No Code AI and Machine Learning: My Program Book

MIT Professional Education, delivered with Great Learning. Cohort January 2026 B.

Prepared for Gilbert Paul Pelletier. Private study copy.

September 2026

Contents

Preface

Part 1. The program at a glance

1.1 Identity card

1.2 MIT faculty behind the content

1.3 How the program was graded

1.4 The people you dealt with, and why it matters for revision

Part 2. Your cohort timeline

2.1 Enrollment and pre work (December 2025 to mid January 2026)

2.2 Core weeks (January 17 to May 4, 2026)

2.3 Completion

Part 3. The curriculum, module by module

Module 0. Pre work: the world of data, and the mathematics behind AI

Module 1. Introduction to the AI and Generative AI landscape

Module 2. Data exploration: structured data, networks, and graphical models

Module 3. Prediction methods: regression

Module 4. Decision systems

Module 5. Project week: machine learning classification. Hotel booking cancellations

Module 6. Recommendation systems

Module 7. Prediction methods: deep learning and neural networks

Module 8. Computer vision methods

Module 9. Project week: prediction methods with deep learning and neural networks (Project 6)

Module 10. Generative AI foundations

Module 11. Business applications of generative AI

Module 12. Ethical and responsible AI

Self paced module A. Data exploration: unstructured data

Self paced module B. Data exploration: temporal data

Part 4. Tools reference

4.1 RapidMiner AI Studio 2025.1 (Altair)

4.2 KNIME Analytics Platform

4.3 Teachable Machine, Ikigai, and generative tools

Part 5. Your projects and portfolio

Part 6. Formula sheet and glossary

6.1 Formula sheet

6.2 Glossary

Part 7. Self test and revision plan

7.1 Twenty questions

7.2 A thirty day plan to go one level deeper

Appendix A. Sources used for this book

Appendix B. Reading list matched to the modules

Preface

This book is a private companion to the program **No Code AI and Machine Learning: Building Data Science Solutions**, offered by MIT Professional Education and delivered online by Great Learning on the Olympus platform. You enrolled on December 29, 2025, started with the January 2026 B cohort (program code MIT NCAIML JAN'26 B), and received your digital certificate on May 26, 2026, followed by the CEU letter on June 3, 2026.

The book has three purposes.

1. Record what the program was: structure, timeline, people, tools, and grading, so you never have to search the portal or your inbox again.
2. Restate the content of every module in a form that suits your background (mathematics degree, options and futures trading, current study of AI and machine learning). Where the course spoke in pictures, this book adds the formula.
3. Give you a revision kit: a formula sheet, a glossary, self test questions, and a plan to go one level deeper on each topic.

How it was built. The Olympus course pages could not be opened from the environment where this book was written, so the reconstruction rests on three kinds of sources: the program emails in your inbox (offer letter, welcome, schedule notices, content release notices, deadlines, certificate), the public program pages and catalog descriptions published by MIT Professional Education and Great Learning, and standard references for the techniques themselves. Every date in Part 2 that comes from an email is marked as confirmed. Dates that are inferred from the 12 week structure are marked as such. Nothing in this book should be treated as an official transcript.

Part 1. The program at a glance

1.1 Identity card

Item	Detail
Program	No Code AI and Machine Learning: Building Data Science Solutions
Institution	MIT Professional Education
Delivery partner	Great Learning (enrollment, payments, platform, support)
Platform	Olympus (olympus.mygreatlearning.com)
Cohort	January 2026 B, code MIT NCAIML JAN'26 B
Program advisor	Amit Jha
Program managers	Juhi (welcome, December 29, 2025), then Ansuman (from January 2, 2026)
Program office email	office.ncai.mit@mygreatlearning.com
Mentor for Mentored Learning Sessions	Aditya Modak
Instructors for optional online classes	Anuj Saboo, Jatin Dawar
Format	Recorded MIT faculty lectures, weekly Mentored Learning Sessions, hands on tool sessions, case studies, quizzes, projects, discussion forum
Duration	12 weeks of core content plus pre work and two self paced modules
Stated workload	11 modules, about 80 study hours, 6 to 12 hours per week
Tools	RapidMiner AI Studio 2025.1 (Altair), KNIME, Teachable Machine, Ikigai, and generative AI tools (ChatGPT, Gemini, NotebookLM, DALL E)
Fee	USD 2,850 total, of which USD 1,000 registration paid December 29, 2025

Item	Detail
Credential	Digital certificate from MIT Professional Education via Accredible (credential.net), issued May 26, 2026
Continuing education	CEU letter issued June 3, 2026 (public program pages state 8 CEUs)

1.2 MIT faculty behind the content

The recorded lectures are built by MIT faculty from the Institute for Data, Systems, and Society (IDSS) and the Department of Electrical Engineering and Computer Science. Public program pages name the following faculty for this program.

- **Devavrat Shah**, professor of AI and decision systems at MIT, director of the Statistics and Data Science Center, and cofounder of Ikigai Labs. His research on recommendation systems and time series is the basis for the recommendation and temporal data modules.
- **Munther Dahleh**, professor and founding director of IDSS. Systems, networks, and decision making.
- **John N. Tsitsiklis**, professor of electrical engineering. Probability, optimization, and the mathematics of learning.
- **Caroline Uhler**, professor at IDSS and the Broad Institute. Graphical models and causal inference, which anchors the "networks and graphical models" part of data exploration.
- **Stefanie Jegelka**, professor of electrical engineering and computer science. Machine learning, deep learning, and optimization.

Mentored sessions were run by Great Learning industry mentors, not by the MIT faculty.

1.3 How the program was graded

From the program office notices:

- **Mentored Learning Sessions** are mandatory and carry an attendance score of 10 marks each. Attendance is recorded only if you join through the Olympus link. Optional online classes and project support sessions carry no attendance score.
- **Weekly course assessments** are short quizzes with a maximum of **two attempts**. The advice was to attempt only after finishing the videos and practice exercises.
- **Course projects** are submitted through Olympus with a deadline. Tool flows were provided for both RapidMiner and KNIME, and you could choose either tool.
- **Project Discussion Forum** contributions were scored (your scores were published on March 28, 2026).

- **Completed projects** were added automatically to your Great Learning ePortfolio.
- **Self paced modules** (Data Exploration for Unstructured Data and for Temporal Data) had optional quizzes that did not affect the grade.

1.4 The people you dealt with, and why it matters for revision

The program had two support layers. Great Learning handled the process (advisor, program manager, WhatsApp, Calendly calls) and the MIT content sat on top. When revising, the useful assets are the MIT lecture videos, the case study videos, the mentor session recordings, and the project problem statements. All of those live on Olympus under your login and remain accessible after completion; alumni retain platform access to the learning content.

Part 2. Your cohort timeline

Dates marked **(confirmed)** come from emails in your inbox. Dates marked *(inferred)* follow from the published 12 week structure and the confirmed anchor points.

2.1 Enrollment and pre work (December 2025 to mid January 2026)

Date	Event
Dec 29, 2025 **(confirmed)**	Advisor call with Amit Jha, offer for registration, payment of USD 1,000, welcome from program manager Juhi, Olympus access opened
Dec 30, 2025 **(confirmed)**	"Start your journey" email: Program Overview course and Pre work course released; orientation announced for Saturday January 17, 2026
Jan 2, 2026 **(confirmed)**	Ansuman introduced as program manager. Note from the platform: case study videos use RapidMiner AI Studio version 2025.1, and learners were told to install that exact version
Jan 4, 2026, Sunday 11:00 EST **(confirmed)**	Optional online class "Intro to the World of AI" with Anuj Saboo (repeat of the December 13 session). Prerequisite: the pre work module "Mandatory Content: Introduction to the World of Data"
Jan 11, 2026, Sunday 12:00 EST **(confirmed)**	Optional online class "The Must Know Mathematics and Statistics Behind AI" with Jatin Dawar
Jan 17, 2026, Saturday **(confirmed)**	Orientation and online class "Intro to the World of AI" with Jatin Dawar. Recording posted the same day. Program formally starts

2.2 Core weeks (January 17 to May 4, 2026)

Week	Dates	Module	Evidence
1	Jan 17 to 23	Introduction to the AI and Generative AI Landscape	Orientation Jan 17 (confirmed)

Week	Dates	Module	Evidence
2	Jan 24 to 30	Data Exploration: Structured Data, Networks, and Graphical Models	*(inferred)*
3	Jan 31 to Feb 6	Prediction Methods: Regression	*(inferred)*
4	Feb 7 to 13	Decision Systems (classification)	*(inferred)*
5	Feb 14 to 20	Project week: Machine Learning Classification. Project "Predicting the cancellation of Hotel Bookings by customers"	Project added to ePortfolio Mar 17 (confirmed)
break	late February	The cohort calendar contained a gap before the recommendation module	*(inferred from the March 7 date)*
6	Mar 1 to 7	Recommendation Systems	Mentored session Sat Mar 7, 13:30 EST, Aditya Modak (confirmed)
7	Mar 8 to 21	Prediction Methods: Deep Learning and Neural Networks	Hands on KNIME session Sun Mar 22, Jatin Dawar (confirmed); mid program feedback survey Mar 10 (confirmed)
8	Mar 22 to 28	Computer Vision Methods	Project forum scores Mar 28 (confirmed); mentored session scheduled Mar 25 (confirmed, topic not in the notice)
9	Mar 29 to Apr 10	Project week and revision week: Project 6, Prediction Methods with Deep Learning and Neural Networks	Revision week notice Apr 9; project due Apr 20, 12:30 EST; support sessions Apr 11 (RapidMiner) and Apr 12 (KNIME) (all confirmed)
10	Apr 11 to 20	Generative AI Foundations	Mentored session Sat Apr 18, Aditya Modak; course assessment due Apr 20, 13:30 (confirmed)

Week	Dates	Module	Evidence
11	Apr 9 to 27	Business Applications of Generative AI (called "Week 9 content" in the office notice)	Content released Apr 9; ninth mentored session Sat Apr 25; assessment due Apr 27, 12:30 EST; project support session Sun Apr 26, 17:00 EDT; course project due May 4, 12:30 EST (all confirmed)
12	Apr 27 to May 4	Ethical and Responsible AI	Course assessment due May 4, 13:30 (confirmed)
self paced	from Apr 23	Data Exploration: Unstructured Data (labelled Week 11) and Data Exploration: Temporal Data (labelled Week 12), optional quizzes	Content release notice Apr 23 (confirmed)

Two remarks on the table. First, the office numbered the Business Applications content "Week 9" and the self paced modules "Weeks 11 and 12", while the mentored session of April 25 was the ninth session of the cohort. The numbering in the notices counts graded modules, not calendar weeks, which is why the two numbering systems disagree. Second, the "Project 6" label on the neural network project tells you that six graded project or assessment slots preceded it.

2.3 Completion

Date	Event
May 26, 2026 **(confirmed)**	Digital certificate issued through Accredible. Your credential page is at credential.net under the ID that appears in the certificate email
Jun 3, 2026 **(confirmed)**	CEU letter sent as a PDF by the program office
Jul 21, 2026 **(confirmed)**	Alumni survey from the MIT PE No Code AI and Machine Learning office

---

Part 3. The curriculum, module by module

Each module below follows the same structure: what the module taught, the mathematics under it (kept at the level you can use, not the level of a proof), how it looked inside the no code tools, the case study or project, and a short list of what to keep.

Module 0. Pre work: the world of data, and the mathematics behind AI

What it taught. Two optional sessions and a self paced course set the vocabulary: data versus information versus decision, descriptive versus predictive versus prescriptive analytics, the difference between AI, machine learning, and deep learning, and how industries went from reporting to prediction. The mathematics session covered the pieces the later modules assume: descriptive statistics, probability, distributions, correlation, basic linear algebra (vectors, matrices, dot products), and the idea of a loss function.

What you already had. With a mathematics degree the content was elementary. The useful part was the mapping between course vocabulary and the mathematics you know:

- "Feature" means a coordinate of the input vector $x \in \mathbb{R}^d$; "label" or "target" means y.
- "Training" means choosing parameters θ to minimize an empirical risk $\frac{1}{n}\sum_i \ell(f_\theta(x_i), y_i)$.
- "Supervised" means y is observed; "unsupervised" means it is not and the goal is structure in x alone.
- "Overfitting" is the gap between empirical risk and expected risk, controlled through validation sets and regularization.

Keep. The train, validation, test split is the single most important discipline in the program. Every later module used it inside the tools.

Module 1. Introduction to the AI and Generative AI landscape

What it taught. A map of the field as of 2025 and 2026: classical machine learning (regression, classification, clustering), deep learning, and the generative wave (large language models, diffusion models for images, and agents that call tools). The lecture placed each of the later modules on that map and introduced the no code philosophy: the workflow is a directed graph of operators (load, clean, transform, model, evaluate, deploy) and the practitioner's job is to choose and connect the operators, not to write them.

The mathematics under it. Not much beyond the definitions above. One idea worth keeping precisely: a generative model estimates a distribution $p(x)$ or $p(x \mid c)$ and samples from it, whereas a discriminative model

estimates $p(y \mid x)$ or a decision function. Everything in modules 2 to 9 is discriminative or descriptive; modules 10 and 11 are generative.

In the tools. First contact with the RapidMiner AI Studio design canvas (repository, operators, ports, process panel, results view) and with the KNIME workflow editor (nodes, node repository, execute, output tables). Both tools were presented as equivalent; project flows were provided in both.

Keep. The operator graph mental model. When you read a Python notebook today, you can translate it back into that graph: each cell is an operator.

Module 2. Data exploration: structured data, networks, and graphical models

What it taught. How to look at a table before modeling it. Summary statistics, missing values, outliers, distributions, correlation, and then three families of unsupervised methods:

1. **Clustering**: K means, K medoids, and Gaussian mixture models.
2. **Dimensionality reduction**: principal component analysis (PCA) and t SNE.
3. **Networks and graphical models**: representing relationships between variables or entities as a graph, and reading conditional independence from the graph structure.

The mathematics under it.

K means minimizes within cluster variance:

$$\min_{S_1,\dots,S_k} \sum_{j=1}^{k} \sum_{x \in S_j} \|x - \mu_j\|^2, \qquad \mu_j = \frac{1}{|S_j|} \sum_{x \in S_j} x.$$

Lloyd's algorithm alternates assignment and mean update and converges to a local minimum. K medoids replaces the mean by an actual data point, which makes it robust to outliers and lets you use any distance. A Gaussian mixture model assumes $p(x) = \sum_j \pi_j \, \mathcal{N}(x; \mu_j, \Sigma_j)$ and fits it by expectation maximization; the soft responsibilities $r_{ij} = \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j) / \sum_l \pi_l \mathcal{N}(x_i; \mu_l, \Sigma_l)$ are what K means hardens to 0 or 1.

PCA finds the orthonormal directions of maximal variance. With centered data matrix X and covariance $C = \frac{1}{n} X^\top X$, the principal components are the eigenvectors of C ordered by eigenvalue, equivalently the right singular vectors of X. The fraction of variance explained by the first m components is $\sum_{i \le m} \lambda_i / \sum_i \lambda_i$. That ratio is the number the tools display as the "scree" or "cumulative variance" plot.

t SNE keeps neighborhoods rather than variance: it converts pairwise distances to probabilities p_{ij} in high dimension and q_{ij} in two dimensions (Student t kernel) and minimizes the Kullback Leibler divergence

$\sum_{i \neq j} p_{ij} \log(p_{ij}/q_{ij})$ by gradient descent. Two consequences the course stressed: distances between t SNE clusters have no metric meaning, and the result depends on the perplexity setting and the random seed.

Graphical models: an undirected graph over variables where a missing edge between i and j encodes conditional independence $X_i \perp X_j \mid X_{\text{rest}}$. For Gaussian data this is read from the zeros of the precision matrix Σ^{-1}, which is the link to Caroline Uhler's research area.

In the tools. RapidMiner operators: Retrieve, Replace Missing Values, Normalize, Detect Outlier, Correlation Matrix, k Means, PCA, and the visualization tab. KNIME nodes: Statistics, Missing Value, Normalizer, k Means, PCA, Scatter Plot.

Case studies. Customer segmentation for marketing, where clusters became personas, and a network view of who buys with whom.

Keep. Standardize before K means and PCA; both are scale dependent. Choose k with the elbow of the within cluster variance and with silhouette scores, then sanity check clusters against business meaning.

Module 3. Prediction methods: regression

What it taught. Predicting a continuous target: simple and multiple linear regression, the meaning of coefficients, goodness of fit, residual diagnostics, regularization (ridge and lasso), and the extension to logistic regression as a bridge to classification.

The mathematics under it.

Ordinary least squares solves $\min_\beta \|y - X\beta\|^2$ with closed form $\hat\beta = (X^\top X)^{-1} X^\top y$. Coefficients are partial effects, holding the other columns fixed, which is why multicollinearity inflates their variance (variance inflation factor). Ridge adds $\lambda \|\beta\|_2^2$ and yields $\hat\beta = (X^\top X + \lambda I)^{-1} X^\top y$, shrinking without zeroing; lasso adds $\lambda \|\beta\|_1$ and produces exact zeros, so it selects features.

Fit metrics: $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$, adjusted R^2 penalizes the number of predictors, and the course preferred RMSE and MAE on a holdout set over any in sample statistic. Logistic regression models $\Pr(y = 1 \mid x) = \sigma(x^\top \beta)$ with $\sigma(z) = 1/(1 + e^{-z})$ and is fit by maximizing the log likelihood $\sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$; each coefficient is a log odds ratio.

A trading reading. A linear regression of returns on factors is exactly this operator with factors as columns. The course's warnings about extrapolation, leverage points, and lookahead in the train test split map directly to overfitting a strategy on in sample data.

In the tools. RapidMiner: Set Role (label), Split Data, Linear Regression, Apply Model, Performance (Regression). KNIME: Partitioning, Linear Regression Learner, Regression Predictor, Numeric Scorer. Both display coefficients, p values, and the residual table.

Case studies. Price or demand prediction from tabular drivers, with a walk through of reading the coefficient table and the residual plot.

Keep. Always compare against a trivial baseline (predict the mean). A model that does not beat it on the holdout is noise.

Module 4. Decision systems

What it taught. Classification as decision making: decision trees, random forests, gradient boosted trees, the confusion matrix, precision, recall, F1, ROC curves and AUC, class imbalance, and how to choose a threshold when the two kinds of error have different costs.

The mathematics under it.

A decision tree splits on the feature and threshold that most reduce impurity. With class proportions p_c at a node, Gini impurity is $1 - \sum_c p_c^2$ and entropy is $-\sum_c p_c \log_2 p_c$; information gain is the parent impurity minus the weighted child impurity. Trees are high variance; a random forest averages many trees grown on bootstrap samples with a random subset of features at each split, which reduces variance without much bias. Gradient boosting instead fits each new tree to the negative gradient of the loss of the current ensemble, so it reduces bias step by step and needs a learning rate and early stopping.

Metrics from the confusion matrix (TP, FP, FN, TN): precision $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$, recall $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$, F1 $= 2PR/(P+R)$. The ROC curve plots recall against false positive rate as the threshold moves; AUC is the probability that a random positive scores above a random negative. When classes are imbalanced, accuracy is misleading and the precision recall curve is the better picture.

Decision theory in one line. If a false negative costs c_{FN} and a false positive costs c_{FP}, predict positive when $\Pr(y=1 \mid x) > c_{FP}/(c_{FP}+c_{FN})$. That threshold rule is the whole reason the module is called "decision systems" rather than "classification".

In the tools. RapidMiner: Decision Tree, Random Forest, Gradient Boosted Trees, Cross Validation, Performance (Classification), and the ROC plot. KNIME: Decision Tree Learner and Predictor, Random Forest Learner, Scorer, ROC Curve.

Keep. Report precision and recall at the threshold the business will actually use, not accuracy at 0.5.

Module 5. Project week: machine learning classification. Hotel booking cancellations

The problem. A hotel chain wants to predict which reservations will be cancelled so that it can manage overbooking, deposits, and pricing. The dataset is the widely used hotel bookings table (lead time, arrival month,

number of nights, market segment, deposit type, customer type, average daily rate, special requests, previous cancellations, and the binary label "is cancelled").

What you did. Loaded and explored the data, treated missing values and outliers, encoded categorical fields, split the data, trained at least a decision tree and a random forest (and typically logistic regression as a baseline), compared them on holdout precision, recall, and F1, chose a model, and wrote business recommendations. The project was added to your ePortfolio on March 17, 2026 under the title "Predicting the cancellation of Hotel Bookings by customers".

The lessons that generalize.

- Lead time and deposit type dominate; long lead times and no deposit are the classic cancellation signature.
- Random forests usually win on this dataset, but the tree's rules are what the hotel manager can act on.
- The cost of a missed cancellation (an empty room) is not equal to the cost of a false alarm (a guest wrongly treated as a risk), so the threshold must be set from those costs, exactly as in Module 4.

Module 6. Recommendation systems

What it taught. How platforms recommend products, films, or content: popularity baselines, content based filtering, collaborative filtering (user based and item based), matrix factorization, and how to evaluate recommendations (precision at k, recall at k, RMSE on held out ratings). Your mentored session on this topic was on Saturday March 7, 2026.

The mathematics under it.

Let $R \in \mathbb{R}^{m \times n}$ be the user by item rating matrix, mostly missing. Item based collaborative filtering predicts $\hat{r}_{ui} = \sum_{j \in N(i)} s_{ij}\, r_{uj} / \sum_{j \in N(i)} |s_{ij}|$ using cosine or Pearson similarity s_{ij} between item columns. Matrix factorization assumes $R \approx U V^{\top}$ with $U \in \mathbb{R}^{m \times k}$, $V \in \mathbb{R}^{n \times k}$ and solves

$$\min_{U,V} \sum_{(u,i) \text{ observed}} (r_{ui} - u_u^{\top} v_i)^2 + \lambda (\|U\|_F^2 + \|V\|_F^2)$$

by alternating least squares or stochastic gradient descent. The low rank assumption is the same one that underlies the singular value decomposition; the difference is that only observed entries enter the loss. Devavrat Shah's own work on collaborative filtering as matrix completion sits directly behind this lecture.

In the tools. RapidMiner has a recommender extension with Item k NN and matrix factorization operators; KNIME builds the same pipeline with similarity and matrix nodes. The course also showed how a simple "customers who bought X also bought Y" rule can be produced from association rules (support, confidence, lift).

Case studies. Streaming and retail recommendation, with a discussion of cold start (new users or items with no history) and of the popularity bias that pure collaborative filtering produces.

Keep. Evaluate on held out interactions per user, not on a random split of ratings; otherwise you leak the user's taste into training.

Module 7. Prediction methods: deep learning and neural networks

What it taught. From logistic regression to a multilayer perceptron: neurons, activation functions, layers, forward propagation, loss, backpropagation, gradient descent and its variants, epochs and batches, overfitting and dropout, and when a neural network beats a tree ensemble on tabular data (rarely) versus on images, text, and audio (usually). The hands on session on KNIME for this module was on Sunday March 22, 2026 with Jatin Dawar.

The mathematics under it.

A network with layers $h^{(l)} = \phi(W^{(l)} h^{(l-1)} + b^{(l)})$ and output $\hat{y} = h^{(L)}$ is trained by minimizing a loss such as cross entropy $-\sum_c y_c \log \hat{y}_c$ (with softmax output $\hat{y}_c = e^{z_c} / \sum_k e^{z_k}$) or squared error. Backpropagation is the chain rule applied layer by layer: $\delta^{(L)} = \nabla_z \ell$, $\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot \phi'(z^{(l)})$, and $\partial \ell / \partial W^{(l)} = \delta^{(l)} h^{(l-1)\top}$. Stochastic gradient descent updates $\theta \leftarrow \theta - \eta \, \hat{\nabla} \ell$ on mini batches; Adam rescales each coordinate by running estimates of the first and second moments of the gradient. ReLU $\phi(z) = \max(0, z)$ avoids the vanishing gradients of sigmoid activations in deep stacks.

In the tools. RapidMiner: Neural Net operator (hidden layer sizes, learning rate, epochs) and the Deep Learning operator from the H2O extension. KNIME: the Keras integration nodes (Keras Network Learner and Executor) shown in the hands on session, plus the simpler RProp MLP Learner.

Keep. Scale inputs, start small, watch training and validation loss together, and stop when validation loss turns up. On tabular business data, a tuned gradient boosted tree is the benchmark a network must beat.

Module 8. Computer vision methods

What it taught. Images as tensors, convolution as learned local filtering, pooling, the architecture of a convolutional neural network, transfer learning from pretrained networks, and image classification and object detection use cases. The no code demonstration used Google's Teachable Machine to train an image classifier from a webcam or uploaded images in minutes.

The mathematics under it.

A convolutional layer computes $(K * I)(i, j) = \sum_{a,b} K(a, b) \, I(i + a, j + b)$ for each learned kernel K; the kernel is shared across positions, which gives translation equivariance and far fewer parameters than a dense layer. Max pooling downsamples by taking the maximum over a window. Stacking convolution, nonlinearity, and pooling builds features from edges to textures to parts to objects. Transfer learning freezes the early layers of a network trained on a large dataset and retrains only the final layers on your images; that is what Teachable Machine does behind the interface, which is why it works with a few dozen examples.

In the tools. Teachable Machine (browser), with export of the trained model, and KNIME's image processing and Keras nodes for a full pipeline.

Case studies. Defect detection in manufacturing, product recognition in retail, and medical imaging as an example where responsible AI questions appear (Module 12).

Keep. Data quality and class balance matter more than architecture at your scale. Look at the misclassified images; they usually reveal a labeling or lighting problem, not a model problem.

Module 9. Project week: prediction methods with deep learning and neural networks (Project 6)

The task. A graded project in which you built a neural network model on a provided business dataset, compared it with the classical models of earlier modules, and reported performance and recommendations. The project was due on April 20, 2026 at 12:30 EST. Optional support sessions were held on April 11 (RapidMiner flow) and April 12 (KNIME flow); you were told to join only the session for the tool you used.

Structure that the graders expected (from the project guide the office referred to):

1. Problem statement and business objective in your own words.
2. Data description and exploratory findings.
3. Preprocessing decisions with justification.
4. Model building, at least one neural network and one comparison model, with hyperparameters stated.
5. Evaluation on the test split with the right metrics.
6. Business insights and recommendations, plus limitations.

Module 10. Generative AI foundations

What it taught. What a large language model is, how it is trained (next token prediction on very large corpora, then instruction tuning and reinforcement from human feedback), tokens and context windows, temperature, hallucination, prompt engineering (role, task, context, format, examples, chain of thought), and the main tools used without code: ChatGPT, Gemini, NotebookLM for grounded question answering on your own documents, and DALL E for images. Your mentored session was on Saturday April 18, 2026 and the course assessment was due April 20.

The mathematics under it.

A language model factorizes $p(w_1, \ldots, w_T) = \prod_t p(w_t \mid w_{<t})$ and is trained by minimizing cross entropy of the next token. The transformer computes each layer with self attention: for queries, keys, and values Q, K, V obtained by linear maps of the token embeddings,

$$\mathrm{Attention}(Q,K,V)=\mathrm{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V,$$

so each token forms a weighted average of the values of all previous tokens with weights that depend on content. Sampling with temperature τ uses $p_i \propto \exp(z_i/\tau)$; $\tau \to 0$ is greedy decoding, higher τ increases diversity and error. Diffusion models for images learn to reverse a gradual noising process, which is why generation starts from noise.

Keep. A prompt is a specification. State role, goal, inputs, constraints, and output format, give one or two examples, and ask the model to show its reasoning when the task is multi step. Verify every fact the model states; fluency is not evidence.

Module 11. Business applications of generative AI

What it taught. Moving from chat to systems: retrieval augmented generation (RAG) so the model answers from your documents, agentic workflows in which a model plans and calls tools, evaluation of generative outputs, cost and latency, and the typical business applications (customer support, document processing, marketing content, code assistance, internal knowledge search). This module carried a quiz (due April 27, 2026) and a course project (due May 4, 2026). A project support session on "Gen AI and Applications" was held on April 26.

The mathematics under it.

RAG turns documents into chunks, embeds each chunk into a vector $e \in \mathbb{R}^d$, and at query time retrieves the chunks whose embeddings have the highest cosine similarity $\cos(q,e) = q^{\top}e/(\|q\|\|e\|)$ with the query embedding; those chunks are placed in the prompt as context. Quality depends on chunk size, overlap, the embedding model, and the number of retrieved chunks k, and it is measured with retrieval recall and with answer faithfulness (does the answer stay inside the retrieved text). An agent is a loop: observe, plan, choose a tool, act, observe again, until a stopping condition; the model's output at each step is parsed into a tool call.

In the tools. NotebookLM and custom GPTs as no code RAG, plus workflow builders for chaining prompts and tools. The project asked for a concrete business use case, a designed prompt or workflow, sample outputs, an evaluation, and a discussion of risks.

Keep. The retrieval step is where most RAG failures happen. Inspect what was retrieved before blaming the model.

Module 12. Ethical and responsible AI

What it taught. Bias and fairness, transparency and explainability, privacy, accountability, safety, and governance frameworks (the EU AI Act, the NIST AI risk management framework, and internal model risk practices). The course assessment was due May 4, 2026.

The mathematics under it. Fairness definitions are constraints on error rates across groups g: demographic parity asks $\Pr(\hat{y} = 1 \mid g)$ to be equal across groups, equalized odds asks true and false positive rates to be equal across groups, and calibration asks $\Pr(y = 1 \mid \hat{p}, g) = \hat{p}$ for every group. An impossibility result states that, when base rates differ across groups, calibration and equalized odds cannot both hold, so fairness is a choice, not a computation. Explainability tools such as feature importance and SHAP values (Shapley values from cooperative game theory applied to features) were shown inside the tools.

Keep. Document the data, the intended use, the metrics by group, and the human override. That documentation is what regulators and auditors ask for.

Self paced module A. Data exploration: unstructured data

What it taught. Text as data: tokenization, stop words, bag of words and TF IDF, sentiment analysis, topic modeling, word embeddings, and a preview of how large language models changed text analytics. Released April 23, 2026 with an optional quiz.

The mathematics under it. TF IDF weights a term t in document d by $\mathrm{tf}(t, d) \times \log(N / \mathrm{df}(t))$. Latent Dirichlet allocation models each document as a mixture of topics and each topic as a distribution over words. Word embeddings place words in $\mathbb{R}^d$ so that cosine similarity reflects co occurrence, which is the same idea as the RAG embeddings of Module 11 applied to single words.

Self paced module B. Data exploration: temporal data

What it taught. Time series: trend, seasonality, stationarity, autocorrelation, moving averages, exponential smoothing, ARIMA style models, and forecasting evaluation with a time ordered split. Released April 23, 2026 with an optional quiz.

The mathematics under it. Autocorrelation at lag h is $\rho(h) = \mathrm{Cov}(x_t, x_{t+h}) / \mathrm{Var}(x_t)$. Simple exponential smoothing is $s_t = \alpha x_t + (1 - \alpha) s_{t-1}$. An AR($p$) model is $x_t = c + \sum_{i=1}^{p} \phi_i x_{t-i} + \varepsilon_t$, and differencing turns a unit root series into a stationary one. For you, this module overlaps with what you know from markets; the main addition is the discipline of walk forward validation, never a random split, on temporal data.

Part 4. Tools reference

4.1 RapidMiner AI Studio 2025.1 (Altair)

- **Why this version.** The case study videos were recorded on 2025.1 and the interface changed in later versions, so the office asked learners to install exactly 2025.1.
- **Canvas vocabulary.** Repository (data and processes), Operators panel, Process canvas, Parameters panel, Results view. Operators connect through ports; the top left "inp" and right "res" ports are the process input and output.
- **A standard supervised flow.** Retrieve, Select Attributes, Replace Missing Values, Nominal to Numerical, Normalize, Set Role (label), Split Data (or Cross Validation), Learner (Decision Tree, Random Forest, Gradient Boosted Trees, Logistic Regression, Neural Net), Apply Model, Performance.
- **A standard unsupervised flow.** Retrieve, Normalize, k Means (or PCA), Cluster Model Visualizer.
- **Auto Model.** A wizard that runs the full flow for you and ranks models; the course used it for quick comparison but asked you to build the manual flow for the projects.

4.2 KNIME Analytics Platform

- **Vocabulary.** Node repository, workflow editor, node status lights (red, yellow, green), output ports, and views.
- **Equivalent nodes.** CSV Reader, Column Filter, Missing Value, One to Many (encoding), Normalizer, Partitioning, learners and predictors, Scorer, Numeric Scorer, ROC Curve, k Means, PCA, Keras Network Learner and Executor.
- **The hands on session** of March 22, 2026 built a neural network flow with the Keras nodes and compared it with a tree model on the same partitioning.

4.3 Teachable Machine, Ikigai, and generative tools

- **Teachable Machine** trains image, sound, or pose classifiers in the browser by transfer learning and exports the model.
- **Ikigai** (cofounded by Devavrat Shah) is a no code platform for tabular data, time series forecasting, and reconciliation used in the recommendation and temporal material.
- **ChatGPT, Gemini, NotebookLM, DALL E** were the generative tools; NotebookLM was the no code way to do retrieval augmented question answering on uploaded documents.

Part 5. Your projects and portfolio

Project	Module	Tool	Deliverable	Status
Predicting the cancellation of Hotel Bookings by customers	5, classification	RapidMiner or KNIME flow, written report	Classification models, metric comparison, recommendations	In ePortfolio since March 17, 2026
Project 6, Prediction Methods with Deep Learning and Neural Networks	9	RapidMiner or KNIME flow	Neural network versus classical models on a business dataset	Due April 20, 2026
Business Applications of Generative AI course project	11	Generative tools, workflow description	Use case, prompts or RAG design, evaluation, risks	Due May 4, 2026
Project Discussion Forum	across modules	Olympus forum	Scored contributions	Scores published March 28, 2026

Your ePortfolio on Olympus holds these projects and can be shared publicly; the sample ePortfolio the advisor sent shows the format.

What came next. After this program you completed the AI Native Professional program (Workflows and Agents for Productivity, June to August 2026) and part of the UT Austin postgraduate program in AI Agents for Business Applications. Those are outside the scope of this book, but the projects from them (MediSure regulatory workflow, GlobalEdge RAG proof of concept, resume shortlisting agent, last mile delivery exception handling) are the direct continuation of Modules 10 and 11 here.

Part 6. Formula sheet and glossary

6.1 Formula sheet

Topic	Formula
Empirical risk	$\hat{R}(\theta) = \frac{1}{n}\sum_i \ell(f_\theta(x_i), y_i)$
OLS	$\hat{\beta} = (X^\top X)^{-1} X^\top y$; ridge $(X^\top X + \lambda I)^{-1} X^\top y$
Logistic	$p = \sigma(x^\top \beta)$, $\sigma(z) = 1/(1+e^{-z})$, coefficient = log odds ratio
Gini, entropy	$1 - \sum_c p_c^2$; $-\sum_c p_c \log_2 p_c$
Precision, recall, F1	$\frac{TP}{TP+FP}$, $\frac{TP}{TP+FN}$, $\frac{2PR}{P+R}$
Cost threshold	predict 1 if $\Pr(y=1 \mid x) > \frac{c_{FP}}{c_{FP}+c_{FN}}$
K means	$\min \sum_j \sum_{x \in S_j} \lVert x - \mu_j \rVert^2$
GMM	$p(x) = \sum_j \pi_j \mathcal{N}(x; \mu_j, \Sigma_j)$, fit by EM
PCA	eigenvectors of $\frac{1}{n} X^\top X$; variance explained $\sum_{i \le m} \lambda_i / \sum_i \lambda_i$
t SNE	minimize $\sum_{i \neq j} p_{ij} \log(p_{ij}/q_{ij})$
Matrix factorization	$\min_{U,V} \sum_{\text{obs}} (r_{ui} - u_u^\top v_i)^2 + \lambda(\lVert U \rVert_F^2 + \lVert V \rVert_F^2)$
Cosine similarity	$q^\top e / (\lVert q \rVert \lVert e \rVert)$
Neural layer	$h^{(l)} = \phi(W^{(l)} h^{(l-1)} + b^{(l)})$; backprop $\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot \phi'(z^{(l)})$
Softmax, cross entropy	$\hat{y}_c = e^{z_c} / \sum_k e^{z_k}$; $\ell = -\sum_c y_c \log \hat{y}_c$
Convolution	$(K * I)(i,j) = \sum_{a,b} K(a,b) I(i+a, j+b)$
Attention	$\mathrm{softmax}(QK^\top / \sqrt{d_k}) V$
Temperature	$p_i \propto \exp(z_i / \tau)$
TF IDF	$\mathrm{tf}(t,d) \log(N / \mathrm{df}(t))$

Topic	Formula
Autocorrelation	$\rho(h) = \mathrm{Cov}(x_t, x_{t+h}) / \mathrm{Var}(x_t)$
AR(p)	$x_t = c + \sum_{i=1}^{p} \phi_i x_{t-i} + \varepsilon_t$

6.2 Glossary

- **Agent.** A model in a loop that plans, calls tools, observes results, and continues until a goal is met.
- **AUC.** Area under the ROC curve; probability that a random positive outranks a random negative.
- **Backpropagation.** Chain rule computation of loss gradients layer by layer.
- **Bagging, boosting.** Averaging models trained on bootstrap samples (variance reduction) versus fitting models sequentially to residual errors (bias reduction).
- **CEU.** Continuing education unit; ten contact hours per unit.
- **Cold start.** A recommendation problem for users or items with no interaction history.
- **Context window.** The maximum number of tokens a language model can attend to at once.
- **Cross validation.** Rotating the validation fold across the data to estimate generalization error with less variance.
- **Embedding.** A dense vector representation of a word, chunk, image, or user learned so that similarity is meaningful.
- **Feature importance.** A ranking of inputs by their contribution to a model's predictions; SHAP values give a per prediction version.
- **Hallucination.** A fluent but false statement by a generative model.
- **Hyperparameter.** A setting chosen before training (depth, learning rate, k, λ), tuned on validation data.
- **Mentored Learning Session (MLS).** The weekly mandatory live session with the Great Learning mentor, scored for attendance.
- **Olympus.** Great Learning's learning management system.
- **Operator, node.** The unit of work in RapidMiner and KNIME respectively.
- **Overfitting.** Low training error with high test error; the model has memorized noise.
- **RAG.** Retrieval augmented generation; retrieve relevant text, then generate with it in the prompt.
- **Regularization.** A penalty on model complexity added to the loss.
- **Stationarity.** Constant mean and autocovariance over time; required by ARIMA style models.
- **Transfer learning.** Reusing a pretrained network and retraining only its last layers.

Part 7. Self test and revision plan

7.1 Twenty questions

1. Why must features be standardized before K means and PCA but not before a decision tree?
2. Write the K means objective and explain why Lloyd's algorithm can stop at a local minimum.
3. What does a Gaussian mixture give you that K means does not?
4. State the PCA solution in terms of the covariance matrix and in terms of the SVD.
5. Why are distances between clusters in a t SNE plot not interpretable?
6. What does a zero in the precision matrix of a Gaussian mean?
7. Derive the OLS estimator from the normal equations. What happens when $X^\top X$ is singular, and how does ridge fix it?
8. Interpret a logistic regression coefficient of 0.69.
9. Compute Gini and entropy for a node with class proportions 0.8 and 0.2.
10. Why does a random forest reduce variance, and why does gradient boosting reduce bias?
11. Given costs $c_{FN} = 5$ and $c_{FP} = 1$, what probability threshold should a classifier use?
12. Why is accuracy misleading on the hotel cancellation data if only 37 percent of bookings cancel?
13. Write the matrix factorization loss for recommendations and explain why only observed entries appear.
14. What is the cold start problem and give one content based remedy.
15. Write the backpropagation recursion for one hidden layer.
16. Why does ReLU help deep networks train?
17. Explain parameter sharing in a convolution and its effect on the number of parameters.
18. What does transfer learning freeze, and why does Teachable Machine need so few examples?
19. Write the attention formula and explain the role of $\sqrt{d_k}$.
20. In a RAG system, name three settings that control retrieval quality and one metric for faithfulness.

7.2 A thirty day plan to go one level deeper

- **Days 1 to 5.** Reproduce the hotel cancellation project in Python (pandas, scikit learn). Match your RapidMiner metrics, then add a cost sensitive threshold and a calibration plot.

- **Days 6 to 10.** Implement K means, PCA, and a Gaussian mixture from scratch with NumPy on the same data. Compare with scikit learn.
- **Days 11 to 15.** Implement matrix factorization by alternating least squares on the MovieLens 100k dataset and evaluate precision at 10 per user.
- **Days 16 to 20.** Build the neural network of Project 6 in PyTorch, log training and validation loss, and beat it with a gradient boosted tree to see the tabular benchmark for yourself.
- **Days 21 to 25.** Fine tune a small pretrained vision model on a two class image set of your own; compare with Teachable Machine.
- **Days 26 to 30.** Build a RAG assistant over your own documents (you already have the commodities RAG assistant and the GlobalEdge proof of concept), then measure retrieval recall and answer faithfulness on twenty hand written questions.

Appendix A. Sources used for this book

Emails in your inbox (Great Learning and MIT PE program office), in date order.

- Dec 29, 2025: brochure and curriculum email; advisor next steps (fee USD 2,850, 11 modules, 80 hours, 6 to 12 hours per week, start January 17, 2026); offer for registration with attached offer letter and brochure; payment receipt; welcome from program manager Juhi.
- Dec 30, 2025: start your journey (Olympus, Program Overview, Pre work, orientation January 17); pre work session notice for January 4.
- Jan 2, 2026: program manager Ansuman; RapidMiner AI Studio 2025.1 requirement.
- Jan 6, 2026: online class notice, Mathematics and Statistics behind AI, January 11.
- Jan 17, 2026: recording of the orientation online class.
- Mar 3, 2026: mentored session notice, Recommendation Systems, March 7.
- Mar 15, 2026: mid program feedback reminder.
- Mar 16, 2026: hands on KNIME session notice, Neural Networks, March 22.
- Mar 17, 2026: ePortfolio, hotel booking cancellation project.
- Mar 28, 2026: project discussion forum scores.
- Apr 9, 2026: revision week, Project 6 deadline, support sessions, Week 9 Business Applications of Generative AI release.
- Apr 15, 2026: mentored session notice, Generative AI Foundations, April 18; Apr 16 course assessment reminder.
- Apr 22 and 23, 2026: mentored session notice for April 25; ninth session notice with assessment and project deadlines; self paced modules Unstructured Data and Temporal Data.
- May 1, 2026: reminders for the Ethical and Responsible AI assessment and the Business Applications assignment, both due May 4.
- May 26, 2026: certificate from Accredible.
- Jun 3, 2026: CEU letter.
- Jul 21, 2026: alumni survey.

Public program descriptions. MIT Professional Education course catalog page for the program; Great Learning program page and brochure summaries; third party listings that reproduce the week by week curriculum (Course Report, Educatly, Northwest Education, Shortcoursesportal, Scribd copies of the brochure).

What could not be verified. The Olympus course pages, the brochure PDF attached to the offer letter, the delivery schedule PDF, and the project guide PDF were not readable from the environment used to write this book. Module

descriptions of weeks 2, 3, 4, 6, 7, 8, and 12 therefore combine the public curriculum with the standard content of those techniques rather than the exact lecture outlines. If you export any of those PDFs from your inbox, this book can be updated against them.

Appendix B. Reading list matched to the modules

- Modules 2 to 7: *An Introduction to Statistical Learning* (James, Witten, Hastie, Tibshirani), chapters on regression, classification, tree methods, and unsupervised learning. Free PDF from the authors.
- Module 6: Koren, Bell, Volinsky, "Matrix Factorization Techniques for Recommender Systems", IEEE Computer, 2009.
- Modules 7 and 8: *Deep Learning* (Goodfellow, Bengio, Courville), chapters 6 and 9. Free online.
- Module 10: Vaswani et al., "Attention Is All You Need", 2017, and the Illustrated Transformer blog post for the pictures.
- Module 11: Lewis et al., "Retrieval Augmented Generation for Knowledge Intensive NLP Tasks", 2020.
- Module 12: NIST AI Risk Management Framework 1.0, and Kleinberg, Mullainathan, Raghavan, "Inherent Trade Offs in the Fair Determination of Risk Scores", 2016.
- Temporal data: Hyndman and Athanasopoulos, *Forecasting: Principles and Practice*, free online.